Any time you define an example or group—that is, any time you use RSpec.describe, context, or it—you can add a hash like the focus: true tag you see here. This hash, known as *metadata*, can contain arbitrary keys and values.

Behind the scenes, RSpec will add metadata of its own, such as last_run_status to indicate whether each spec passed or failed the last time it ran.

You can filter examples directly from the command line using the --tag option. For example, if RSpec didn't already have an --only-failures command-line option, you could have gotten the same behavior like so:

```
$ rspec --tag last_run_status:failed
Run options: include {:last_run_status=>"failed"}
F

« truncated »

1 example, 1 failure

Failed examples:

rspec ./spec/coffee_spec.rb:29 # A cup of coffee with milk costs $1.25
```

Similarly, you could pass --tag focus to run just the focused specs, but instead we configured RSpec to do so by default.

Before continuing, don't forget to change fcontext back to context. The whole idea of focusing is to filter specs *temporarily*.

Marking Work in Progress

In BDD, you're typically working on getting just one spec at a time to pass. Trying to tackle too many features at once leads to the kinds of complicated, untestable designs that BDD seeks to avoid.

On the other hand, it can be extremely productive to *sketch out* several examples in a batch. You're thinking of all the things a software component needs to do, and you want to capture the ideas so you don't forget anything. RSpec supports this workflow really well, through *pending* examples.

Starting With the Description

While you were writing the coffee spec earlier, you may have been thinking ahead to other properties of coffee with milk: it's lighter in color, cooler, and so on. While these behaviors are on your mind, go ahead and add them inside the with milk context as empty examples:

02-running-specs/09/spec/coffee_spec.rb

```
it 'is light in color'
it 'is cooler than 200 degrees Fahrenheit'
```